



Python Core Refresher

01

Data Structures

Integer ,float , Boolean , List, Tuple, Set, Dict, String

02

Comprehensions , Iterator & Generators

Iterators, yield, lazy evaluation

03

Decorators

Functions as first-class, stacking

04

Type Hints & Error Handling

Annotations, try/except, custom errors

05

OOP Concepts

Classes, Object , Encapsulation, Inheritance, Abstraction , Polymorphism, MRO

06

Dataclasses

Auto-generated methods, frozen, field()

07

Pydantic

Validation, schemas, nested models

SECTION 01



Data Structures

12 List — Ordered, Mutable, Allows Duplicates

📌 When to Use: integer (int) data type is used to store whole numbers like 10, -5, or 0 and it is immutable in python

```
a = 10
b = 10

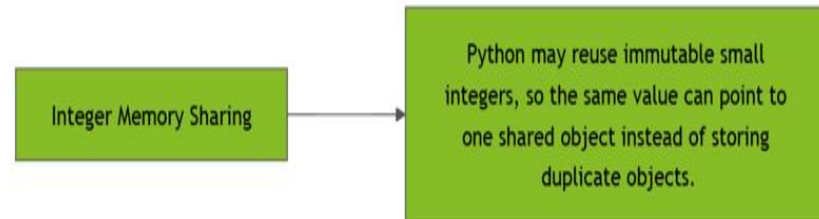
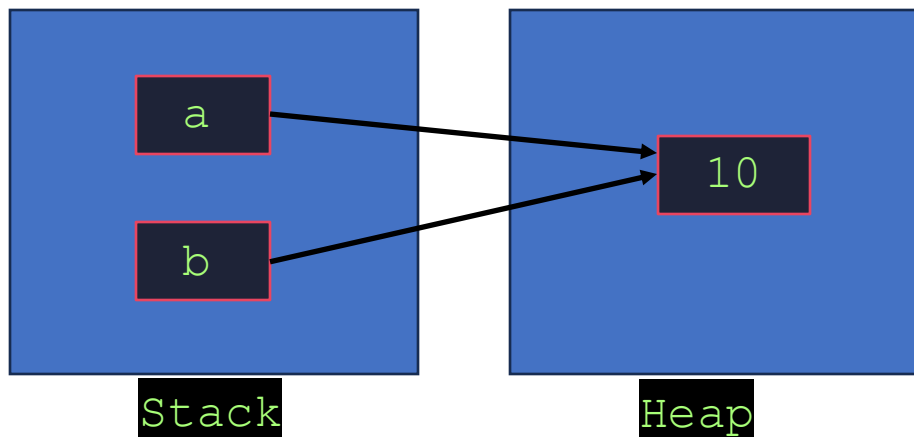
print(a is b)    # True
print(a == b)    # True
```

1. `==` checks value equality

`a == b` means: “Do both variables have the same value?”

2. `is` checks object identity

`a is b` means: “Do both variables point to the exact same object in memory?”



12 List — Ordered, Mutable, Allows Duplicates

📌 When to Use: Maintain ordered collection | Duplicates allowed | Need indexing | Data changes over time (mutable)

```
fruits = ['apple', 'banana', 'apple']
fruits.append('mango')           # →
['apple', 'banana', 'apple', 'mango']
fruits[1]                        # → 'banana' (indexing)
fruits[0:2]                      # → ['apple', 'banana']
(slicing)
```

- ✓ Ordered (maintains insertion order)
- ✓ Mutable (can change after creation)
- ✓ Allows Duplicate values
- ✓ Supports Indexing & Slicing
- ✓ Can hold mixed data types

Method	Example	Description
<code>append(x)</code>	<code>lst.append('kiwi')</code>	Add single item at end
<code>extend([x])</code>	<code>lst.extend(['a','b'])</code>	Add multiple items at end
<code>insert(i,x)</code>	<code>lst.insert(1,'grape')</code>	Insert at specific index
<code>remove(x)</code>	<code>lst.remove('apple')</code>	Remove first occurrence
<code>pop(i)</code>	<code>lst.pop(0)</code>	Remove & return item at index
<code>sort()</code>	<code>lst.sort(reverse=True)</code>	Sort list in-place
<code>reverse()</code>	<code>lst.reverse()</code>	Reverse list in-place
<code>index(x)</code>	<code>lst.index('mango')</code>	Return index of first match
<code>count(x)</code>	<code>lst.count('apple')</code>	Count occurrences
<code>copy()</code>	<code>lst2 = lst.copy()</code>	Shallow copy of list
<code>clear()</code>	<code>lst.clear()</code>	Remove all items

Tuple — Ordered, Immutable, Allows Duplicates

 When to Use: Fixed data that should NOT change | Return multiple values from a function | Dict keys (immutable) | Faster than list

```
point = (10, 20)
name, age = ('Alice', 25)    # tuple unpacking
coords = (1.0, 2.0, 3.0)
print(point[0])              # → 10
# point[0] = 5               # ❌ TypeError:
                             # immutable!
```

- ✅ Ordered (maintains sequence)
- ❌ Immutable (cannot modify after creation)
- ✅ Allows Duplicate values
- ✅ Can be used as dictionary key
- ✅ Faster than list (less memory)
- ✅ Supports Indexing & Slicing

Method	Example	Description
<code>count(x)</code>	<code>t.count(10)</code>	Count occurrences of a value
<code>index(x)</code>	<code>t.index(20)</code>	Return index of first occurrence
<code>len(t)</code>	<code>len(point)</code>	Number of elements in tuple
<code>t1 + t2</code>	<code>(1, 2) + (3, 4) → (1, 2, 3, 4)</code>	Concatenate two tuples
<code>t * n</code>	<code>(1, 2) * 3 → (1, 2, 1, 2, 1, 2)</code>	Repeat tuple n times
<code>in operator</code>	<code>10 in point → True</code>	Check if value exists
<code>Unpacking</code>	<code>x, y = point</code>	Unpack into individual variables
<code>Named Tuple</code>	<code>from collections import namedtuple</code>	Tuple with named fields

Set — Unordered, Mutable, NO Duplicates

📌 When to Use: Remove duplicates | Membership testing (very fast $O(1)$) | Mathematical set operations | Unique collections

```
s = {1, 2, 3, 2, 1}      # → {1, 2, 3}  (auto removes
dups)
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
A | B  # Union      → {1, 2, 3, 4, 5, 6}
A & B  # Intersect  → {3, 4}
A - B  # Diff       → {1, 2}
```

- ✗ Unordered (no indexing)
- ✓ Mutable (add/remove items)
- ✗ NO Duplicates allowed
- ✓ Extremely fast membership test
- ✓ Mathematical set operations
- ⚠ Elements must be hashable

Method	Example	Description
<code>add(x)</code>	<code>s.add(5)</code>	Add a single element
<code>update([x])</code>	<code>s.update([6,7])</code>	Add multiple elements
<code>remove(x)</code>	<code>s.remove(3)</code>	Remove element (raises error if absent)
<code>discard(x)</code>	<code>s.discard(99)</code>	Remove element (no error if absent)
<code>pop()</code>	<code>s.pop()</code>	Remove & return arbitrary element
<code>union()</code>	<code>A.union(B)</code> or <code>A B</code>	All unique elements from both
<code>intersection()</code>	<code>A.intersection(B)</code> or <code>A & B</code>	Common elements
<code>difference()</code>	<code>A.difference(B)</code> or <code>A - B</code>	In A but not in B
<code>issubset()</code>	<code>A.issubset(B)</code>	Check if $A \subseteq B$
<code>frozenset()</code>	<code>fs = frozenset({1,2,3})</code>	Immutable version of set

★ When to Use: Text processing | User input | File I/O | Templates | Pattern matching

```
s = 'Hello World'
s.upper()           # → 'HELLO WORLD'
s.split(' ')        # → ['Hello', 'World']
f"Name: {'Alice'}, Age: {25}" # f-string
's' in s            # → True
```

- ✗ Immutable (cannot change in-place)
- ✓ Ordered sequence of characters
- ✓ Supports indexing & slicing
- ✓ Rich built-in methods
- ✓ f-strings for formatting (3.6+)
- ✓ Multi-line with triple quotes

Method	Example	Description
<code>upper()</code> / <code>lower()</code>	<code>"hi".upper()</code>	Case conversion
<code>strip()</code>	<code>" hi ".strip()</code>	Remove leading/trailing whitespace
<code>split(sep)</code>	<code>"a,b".split(",")</code>	Split into list
<code>join(iter)</code>	<code>",".join(["a","b"])</code>	Join iterable into string
<code>replace(a,b)</code>	<code>"hi".replace("i","o")</code>	Replace occurrences
<code>find(sub)</code>	<code>"hello".find("ll")</code>	Return index of substring (-1 if not found)
<code>startswith()</code>	<code>"hello".startswith("he")</code>	Check beginning
<code>endswith()</code>	<code>"hello".endswith("lo")</code>	Check ending
<code>format()</code>	<code>"{} {}".format(1,2)</code>	String formatting
<code>count(sub)</code>	<code>"aabaa".count("a")</code>	Count occurrences of substring
<code>isdigit()</code>	<code>"123".isdigit()</code>	Check if all chars are digits

📌 When to Use: Key-value mapping | Fast lookup by key $O(1)$ | Config/settings | JSON-like data | Counting occurrences

```
student = {'name': 'Alice', 'age': 25, 'grade': 'A'}
student['age']          # → 25
student['city'] = 'Chennai' # add new key
student.get('phone', 'N/A') # → 'N/A' (safe access)
for k, v in student.items(): print(k, v)
```

- ✓ Ordered by insertion (Python 3.7+)
- ✓ Mutable (add/edit/delete keys)
- ✓ Keys must be unique & hashable
- ✓ Values can be any type
- ✓ Very fast key lookup $O(1)$
- ⚠ Key access raises `KeyError` if missing

Method	Example	Description
<code>get(k, def)</code>	<code>d.get('phone', 'N/A')</code>	Safe key access with default
<code>keys()</code>	<code>d.keys()</code>	View of all keys
<code>values()</code>	<code>d.values()</code>	View of all values
<code>items()</code>	<code>d.items()</code>	View of all key-value pairs
<code>update(d2)</code>	<code>d.update({'city': 'CHN'})</code>	Merge another dict
<code>pop(k)</code>	<code>d.pop('grade')</code>	Remove key and return value
<code>setdefault()</code>	<code>d.setdefault('city', 'N/A')</code>	Set key only if missing
<code>fromkeys()</code>	<code>dict.fromkeys(['a', 'b'], 0)</code>	Create dict from keys
<code>copy()</code>	<code>d2 = d.copy()</code>	Shallow copy of dict
<code>del d[k]</code>	<code>del student['age']</code>	Delete specific key
<code>in operator</code>	<code>'name' in d</code>	Check key existence

Data Structure Comparison — Which One When?

Data Structures

Property	List	Tuple	Set	Dict	String
Ordered	✓	✓	✗	✓ (3.7+)	✓
Mutable	✓	✗	✓	✓	✗
Duplicates	✓	✓	✗	Keys: ✗	✓
Indexing	✓	✓	✗	By Key	✓
Syntax	[]	()	{ }	{ k:v }	" "
Lookup Speed	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Memory	Medium	Low	Medium	High	Low
Best Use	Ordered data	Fixed data	Unique items	Key-Value	Text

Quick Decision Guide

 Ordered + changeable → LIST |  Fixed/constant data → TUPLE

 No duplicates + fast search → SET |  Key-value lookup → DICT

 Text data / processing → STRING |  Dict key / hashable → TUPLE or FROZENSET

QUIZ TIME!

Data Structures — Test Your Knowledge!

Q1: Which data structure allows duplicates AND is immutable? → List / Tuple / Set / Dict

Q2: What is the time complexity of membership testing in a Set? → $O(1)$ / $O(n)$ / $O(\log n)$ / $O(n^2)$

Q3: You want to store (lat, long) coordinates that should never change — which is best? → List / Tuple / Set / Dict

Q4: What happens when you access `dict["missing_key"]` if the key does not exist? → Returns None / Returns False / Raises KeyError / Returns empty string

SECTION 02



Comprehensions & Generators

Syntax: `[ expression for item in iterable if condition ]`

❌ Normal Function (Verbose)

```
# Get squares of even numbers 1-10
result = []
for num in range(1, 11):
    if num % 2 == 0:
        result.append(num ** 2)
print(result)  # [4,16,36,64,100]
```

✅ Comprehension (Pythonic)

```
# Same result in ONE line!
result = [num**2 for num in range(1,11)
          if num % 2 == 0]
print(result)  # [4,16,36,64,100]
```

All Comprehension Types — Syntax

List

```
[ x*2 for x in lst ]
```

Returns a list

Dict

```
{ k: v for k, v in d.items() }
```

Returns a dictionary

Set

```
{ x for x in lst }
```

Returns a set (no dups)

Generator

```
( x*2 for x in lst )
```

Returns a lazy iterator (memory efficient)



Iterator & Generator — Lazy Evaluation

Comprehensions & Generators



ITERATOR — What & Why

An **Iterator** is any object that implements:

`__iter__()` → Returns the iterator object itself

`__next__()` → Returns the next value (raises `StopIteration` when done)

```
nums = [1, 2, 3]
it = iter(nums)          # create iterator
print(next(it))          # → 1
print(next(it))          # → 2
print(next(it))          # → 3
# next(it)               # → StopIteration!
```



GENERATOR — Memory Efficient

A **Generator** is a special function using `yield`.

It produces values

ONE AT A TIME — values are NOT stored in memory. Perfect for large datasets!

```
def count_up(n):
    for i in range(n):
        yield i          # pause & return i

gen = count_up(3)
print(next(gen))         # → 0
print(next(gen))         # → 1
```



Real-World Generator Example — Reading Large Files Line by Line

Generator Methods

```
def read_large_file(filepath):
    """Generator: reads ONE line at a time — won't crash on 10GB
    files!"""
    with open(filepath, 'r') as f:
        for line in f:
            yield line.strip()    # yields one line, pauses

for line in read_large_file('bigdata.csv'):    # memory: only 1
    line at a time
    process(line)
```

List: `[x for x in range(10M)]`

→ Stores ALL 10M items in RAM 🗄️

Generator: `(x for x in range(10M))`

→ Produces ONE item at a time ✅

QUIZ TIME!

Comprehensions & Generators — Test Your Knowledge!

Q1: What does `[x for x in range(5) if x%2==0]` produce? → `[0,2,4]` / `[1,3]` / `[0,1,2,3,4]` / Error

Q2: What keyword makes a function a generator? → `return` / `yield` / `async` / `generate`

Q3: A generator is better than a list when: → Speed matters / Data is huge & you read once / Need indexing / Need sorting

Q4: What exception is raised when a generator is exhausted? → `IndexError` / `ValueError` / `StopIteration` / `GeneratorExit`

SECTION 03



Decorators



Decorators — What & Why

 **Definition:** A decorator is a function that WRAPS another function to ADD extra behaviour — WITHOUT modifying the original function's code.

Why Do We Need Decorators?

 Reusability — add same behavior to many functions

 Clean Code — no repetition (DRY principle)

 Logging/Timing without modifying function

 Authentication & access control

 Caching expensive function results

Decorator Flow:

@my_decorator applied



say_hello → wrapper()



wrapper runs BEFORE code



original func() called



wrapper runs AFTER code

Simple Example — Add Logging

```
def my_decorator(func):  
    def wrapper():  
        print("Before function")  
        func()           # call original  
        print("After function")  
    return wrapper  
  
@my_decorator  
def say_hello():  
    print("Hello!")  
  
say_hello()  
# Output:  
# Before function  
# Hello!  
# After function
```



Stacking Decorators — Flow When 2 Decorators Applied

Decorators

```
def decorator_A(func):
    def wrapper():
        print("A: Before")
        func()
        print("A: After")
    return wrapper

def decorator_B(func):
    def wrapper():
        print("B: Before")
        func()
        print("B: After")
    return wrapper

@decorator_A           # ← applied SECOND (outer)
@decorator_B           # ← applied FIRST (inner)
def greet():
    print("Hello!")

greet()

greet = decorator_A(decorator_B(greet))

Execution order when calling greet():

decorator_A.wrapper: prints "A: Before" calls
decorator_B.wrapper: prints "B: Before" calls
greet(): prints "Hello!" prints "B: After" prints
"A: After"
```

✓ Output

```
A: Before
B: Before
Hello!
B: After
A: After
```

🔄 Execution Flow (LIFO Order)

- | | | |
|---|------------------|-------------------|
| 1 | Call greet() | Entry point |
| 2 | A wrapper BEFORE | Print "A: Before" |
| 3 | B wrapper BEFORE | Print "B: Before" |
| 4 | greet() runs | Print "Hello!" |
| 5 | B wrapper AFTER | Print "B: After" |
| 6 | A wrapper AFTER | Print "A: After" |

QUIZ TIME!

Decorators — Test Your Knowledge!

Q1: A decorator is applied using which symbol? → @ / # / \$ / &

Q2: When two decorators @A and @B are stacked, which runs first? → A / B / Both simultaneously / Depends on arguments

Q3: @A @B def f() is equivalent to: → $f = A(B(f))$ / $f = B(A(f))$ / A and B wrap together / $f = AB(f)$

Q4: Which is a common real use of decorators? → Adding logging without changing function / Changing the function name / Making functions private / Converting to async

SECTION 04



Type Hints & Error Handling



Type Hints — Documentation, Not Enforcement

⚠ IMPORTANT: Type hints are HINTS — Python does NOT enforce them at runtime. Wrong types give IDE warnings, NOT errors. Use `mypy` for static type checking.

```
# Basic type hints
def greet(name: str) -> str:
    return f'Hello, {name}'

# Wrong type — Python RUNS fine, IDE shows warning
greet(123)      # ← IDE: warning! No runtime error.

# Collection types
from typing import List, Dict, Optional, Union, Tuple
def process(items: List[int]) -> Dict[str, int]:
    return {'sum': sum(items), 'count': len(items)}

# Optional (value or None)
def find_user(id: int) -> Optional[str]:
    return 'Alice' if id == 1 else None

# Union (multiple allowed types)
def stringify(val: Union[int, float]) -> str:
    return str(val)
```

✅ Python 3.9+ Simplified Syntax:

```
list[int]  dict[str, int]  tuple[int, str]
(No need to import from typing)
```

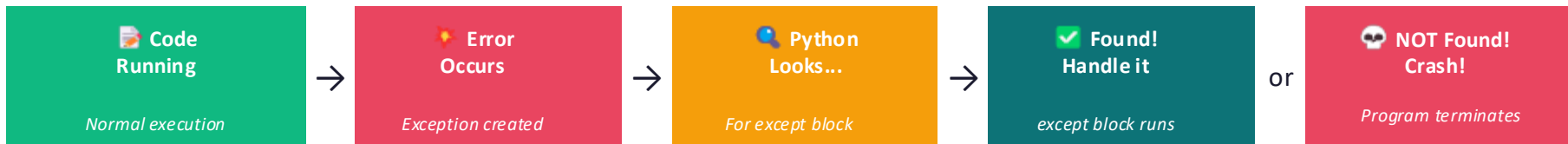
Common Type Annotations

<code>int</code>	Whole number
<code>float</code>	Decimal number
<code>str</code>	Text string
<code>bool</code>	True or False
<code>List[int]</code>	List of ints
<code>Dict[str, int]</code>	String→int map
<code>Optional[str]</code>	str or None
<code>Union[int, str]</code>	int or str
<code>Tuple[int, str]</code>	Fixed-length tuple
<code>Any</code>	Any type allowed
<code>Callable</code>	Function type
<code>set[str]</code>	Set of strings (3.9+)

Error Handling — Why & How (Flow Diagram)

Error Handling

What Happens When an Error Occurs in Python?



try / except / else / finally — Complete Structure

```
try:
    result = 10 / int(input('Enter number: '))
except ZeroDivisionError:
    print('Cannot divide by zero!')    # specific error
except ValueError as e:
    print(f'Invalid input: {e}')      # another specific error
except Exception as e:
    print(f'Unexpected error: {e}')   # catch-all
else:
    print(f'Result: {result}')         # runs if NO error
finally:
    print('This ALWAYS runs!')        # cleanup code
```

try:

Code that might raise an error. If error occurs, jumps to except.

except:

Handles specific error type. Can have multiple except blocks.

else:

Runs ONLY if try block had NO exceptions. Optional.

finally:

ALWAYS runs regardless. Good for cleanup (close files, DB connections).

Exception Types & Custom Exceptions

Built-in Exception Hierarchy

BaseException	<i>Root of all exceptions</i>
Exception	<i>Base for regular errors</i>
ValueError	<i>Wrong value type</i>
TypeError	<i>Wrong data type</i>
ZeroDivisionError	<i>Divide by zero</i>
IndexError	<i>List index out of range</i>
KeyError	<i>Dict key missing</i>
AttributeError	<i>Object has no attribute</i>
FileNotFoundError	<i>File does not exist</i>
ImportError	<i>Module import failed</i>
StopIteration	<i>Iterator exhausted</i>



Creating Custom Exceptions

```
# Define custom exception
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(
            f'Need ₹{amount}, have ₹{balance}'
        )

# Use it
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance,
            amount)
    return balance - amount

try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print(e)    # Need ₹500, have ₹100
```

raise & assert Keywords

```
raise ValueError('Bad value')    # manually raise
assert x > 0, 'x must be positive' # debug check
```

QUIZ TIME!

Type Hints & Error Handling — Test Your Knowledge!

Q1: Type hints in Python cause a RuntimeError when wrong type is passed? → True / False (only IDE warnings) / Depends on Python version / True only in 3.10+

Q2: Which block ALWAYS runs in a try/except structure? → try / except / else / finally

Q3: Which exception is raised when you access a missing dictionary key? → IndexError / ValueError / KeyError / AttributeError

Q4: To create a custom exception, you should inherit from: → object / BaseClass / Exception / Error

SECTION 05



Object Oriented Programming

A CLASS is a blueprint. An OBJECT is an instance created from that blueprint. Encapsulation = bundling data + methods + hiding internal details.

```
class BankAccount:
    bank_name = 'MyBank'          # class variable (shared)

    def __init__(self, owner, balance):
        self.owner = owner        # public attribute
        self._account_no = '12345' # _ = protected
    (convention)
        self.__balance = balance  # __ = private (name mangling)

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):         # getter method
        return self.__balance

acc = BankAccount('Alice', 1000)
acc.deposit(500)
print(acc.get_balance())         # → 1500
print(acc._balance)              # ❌ AttributeError!
print(acc._BankAccount__balance) # ✅ name mangling access
```

Access Levels in Python

name	Public	Accessible anywhere
<code>_name</code>	Protected	Convention: internal use only
<code>__name</code>	Private	Name mangling applied



Name Mangling Explained:

`__balance` in class BankAccount

becomes:

`_BankAccount__balance`

Python renames it to prevent accidental override in subclasses.

Key Dunder Methods

<code>__init__</code>	→ Constructor
<code>__str__</code>	→ <code>print(obj)</code> output
<code>__repr__</code>	→ <code>repr(obj)</code> output
<code>__len__</code>	→ <code>len(obj)</code>
<code>__eq__</code>	→ <code>obj1 == obj2</code>



Inheritance — Reuse & Extend

OOP

Inheritance lets a child class REUSE and EXTEND a parent class. The child gets all parent methods/attributes + can add its own.

```
class Animal:                                # Parent class
    def __init__(self, name):
        self.name = name
    def speak(self):                          # method to override
        return f'{self.name} makes a sound'
    def breathe(self):
        return 'Breathes oxygen'

class Dog(Animal):                            # Single Inheritance
    def speak(self):                        # Override parent
        method
        return f'{self.name} says Woof!'
    def fetch(self):                         # New method
        return f'{self.name} fetches the ball'

class GuideDog(Dog):                         # Multi-level
Inheritance                                Inheritance
    def guide(self):
        return f'{self.name} guides the blind'

d = Dog('Rex')
print(d.speak())        # → Rex says Woof!      (overridden)
print(d.breathe())      # → Breathes oxygen      (from Animal)
print(d.fetch())        # → Rex fetches...       (Dog's own)
```

Types of Inheritance

Single

A → B

One parent, one child

Multi-level

A → B → C

Chain of inheritance

Multiple

A, B → C

Child has two+ parents

Hierarchical

A → B, C

One parent, many children

Hybrid

Mix of above *Combination of types*

```
# super() — call parent method
class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name)    # call
Animal.__init__
        self.color = color
```

 **MRO** = When a method is called, Python searches classes in a specific ORDER (left-to-right, depth-first with C3 algorithm). Use `ClassName.__mro__` to see the order.

```
class A:
    def hello(self): return 'A'

class B(A):
    def hello(self): return 'B'

class C(A):
    def hello(self): return 'C'

class D(B, C):    # Multiple inheritance
    pass

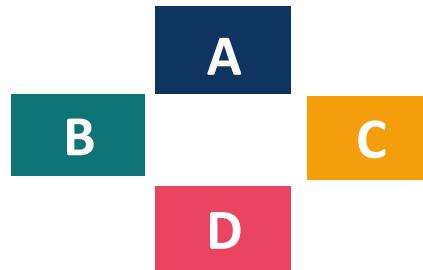
d = D()
print(d.hello())    # → 'B'    (MRO order: D→B→C→A)
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class
'object'>)
print(D.mro())
# [D, B, C, A, object]
```

✅ Use `mro()` to debug:

```
print(D.mro())
print(D.__mro__)
```

Useful when you have multiple inheritance and need to know which method will be called.

MRO Diamond Diagram



MRO: D → B → C → A → object

MRO Rules

- 1 Search starts from the class itself (D)
- 2 Left-to-right for multiple parents (B before C)
- 3 Depth before breadth (but respects linearization)
- 4 Every class appears only once
- 5 object class is always last



ABSTRACTION — Hide Complexity

Show only what is necessary, hide internal implementation. Use ABC module for abstract classes.

```
from abc import ABC, abstractmethod

class Shape(ABC):          # Abstract class
    @abstractmethod
    def area(self):
        pass # MUST implement
    def describe(self):
        return f'Area: {self.area()}'

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r**2

# Shape()      ✗ Cannot instantiate abstract!
c = Circle(5)
print(c.describe())    # → Area: 78.5
```



POLYMORPHISM — Many Forms

Same method name, different behavior per class. One interface, many implementations.

```
class Dog:
    def speak(self): return 'Woof!'

class Cat:
    def speak(self): return 'Meow!'

class Duck:
    def speak(self): return 'Quack!'

# Polymorphism in action
animals = [Dog(), Cat(), Duck()]
for a in animals:
    print(a.speak()) # each behaves differently
```



Duck Typing — "If it walks like a duck and quacks like a duck, it IS a duck"

```
class Parrot:
    def speak(self): return 'Squawk!'

# Python doesn't care about TYPE — only that
# speak() exists!
def make_it_speak(animal): # no type
    restriction!
    print(animal.speak())

make_it_speak(Dog()) # Woof! (Duck typing
works!)
make_it_speak(Parrot()) # Squawk! (no inheritance
needed)
```



Difference:

Polymorphism = same method name in RELATED classes (via inheritance)

Duck Typing = Python checks if the method EXISTS regardless of class

QUIZ TIME!

OOP — Test Your Knowledge!

Q1: What does `__name__` become after name mangling in class `MyClass`? → `__MyClass__name` / `_MyClass__name` / `__name__` / `MyClass__name`

Q2: Which module provides ABC for abstract classes? → `abstract` / `abc` / `interface` / `base`

Q3: In `D(B, C)`, what is the MRO order? → `D→C→B→A→object` / `D→B→C→A→object` / `D→A→B→C→object` / `B→C→D→A→object`

Q4: Duck typing means: → Only ducks can be used / Class must inherit from `Duck` / Method existence matters, not class type / Type checking at runtime

SECTION 06



Data classes

💡 **Problem:** Writing `__init__`, `__repr__`, `__eq__` manually is repetitive. `@dataclass` decorator auto-generates all this!

❌ WITHOUT `@dataclass` (verbose)

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade
    def __repr__(self):
        return
f'Student({self.name},{self.age},{self.grade}) '
    def __eq__(self, other):
        return (self.name == other.name and
                self.age == other.age)
```

✅ WITH `@dataclass` (clean!)

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    age: int
    grade: str = 'A' # default value
    scores: list = field(default_factory=list)

s1 = Student('Alice', 20)
s2 = Student('Alice', 20)
print(s1)           # → Student(name='Alice'...)
print(s1 == s2)     # → True (auto __eq__!)
```

Key Dataclass Features

`@dataclass`Auto `__init__`, `__repr__`, `__eq__`

Basic dataclass

`@dataclass(frozen=True)`

Immutable (like tuple)

Cannot change attributes after creation

`@dataclass(order=True)`Auto `<`, `>`, `<=`, `>=` comparisons

Compare objects by field values

`field(default_factory=list)`

Mutable default values

Avoids sharing default mutable objects

SECTION 07



Pydantic — Data Validation

💡 **Pydantic** = Data validation library that uses Python type hints to validate, parse & serialize data at RUNTIME. Unlike type hints alone, Pydantic **DOES raise errors for wrong types!**

✗ Type Hints Only — No Validation

```
def create_user(name: str, age: int):  
    return {'name': name, 'age': age}  
  
create_user('Alice', 'not_a_number')  
# ← NO ERROR! Python just runs it.  
# age is still a string 'not_a_number'
```

✓ Pydantic — Real Validation

```
from pydantic import BaseModel  
  
class User(BaseModel):  
    name: str  
    age: int  
  
User(name='Alice', age='30') # ✓ auto-converts  
User(name='Alice', age='abc') # ✗ ValidationError!  
# Pydantic ENFORCES types at runtime
```

Pydantic Key Features

```
from pydantic import BaseModel, Field, validator
from typing import Optional, List

class Product(BaseModel):
    name: str = Field(..., min_length=2, max_length=50)
    price: float = Field(..., gt=0)      # must be > 0
    discount: Optional[float] = 0.0
    tags: List[str] = []

    @validator('price')
    def price_must_be_valid(cls, v):
        if v > 100000:
            raise ValueError('Price too high!')
        return round(v, 2)

p = Product(name='Laptop', price=49999.5, tags=['tech'])
print(p.dict())      # → {'name': 'Laptop', 'price': 49999.5, ...}
print(p.json())      # → JSON string
Product(name='X', price=-100)  # ✗ ValidationError: price > 0
```

Where Pydantic Shines



FastAPI

Request/Response body validation



Config

App settings from env vars



Database

Input validation before DB insert



API Client

Parse & validate API responses

```
# NESTED MODELS
class Address(BaseModel):
    street: str
    city: str
    pincode: str

class Employee(BaseModel):
    id: int
    name: str
    address: Address          # ← nested model!
    skills: List[str] = []

# Usage
emp = Employee(
    id=1, name='Alice',
    address={'street': '10 MG Rd', 'city': 'Chennai',
            'pincode': '600001'},
    skills=['Python', 'FastAPI']
)
print(emp.address.city)      # → Chennai
print(emp.dict())           # nested dict
print(emp.json(indent=2))    # pretty JSON

# PARSE FROM JSON
emp2 = Employee.parse_raw(json_string)
# PARSE FROM DICT
emp3 = Employee.parse_obj(some_dict)
```

⚙️ Pydantic Settings

```
from pydantic import BaseSettings

class AppConfig(BaseSettings):
    db_url: str
    debug: bool = False
    secret_key: str

    class Config:
        env_file = '.env'

config = AppConfig()
print(config.db_url)  # reads from env!
```

Key Pydantic Model Methods

<code>model.dict()</code>	Convert to Python dictionary
<code>model.json()</code>	Convert to JSON string
<code>Model.parse_raw(s)</code>	Parse from JSON string
<code>Model.parse_obj(d)</code>	Parse from Python dict
<code>model.copy(update={})</code>	Clone with field updates
<code>Model.schema()</code>	Get JSON Schema definition

Pydantic vs Dataclass

Feature	Dataclass	Pydantic
Runtime validation	✗	✓
Type correction	✗	✓
JSON serialization	✗	✓
Nested models	⚠ manual	✓ auto
Custom validators	✗	✓
Performance	⚡ faster	🐢 slower
When to use	Internal data	API/external data

QUIZ TIME!

Pydantic — Test Your Knowledge!

Q1: Pydantic validates data at: → Compile time / Import time / Runtime / Only in tests

Q2: `User(name="Alice", age="25")` in Pydantic — what happens? → `TypeError` / `ValidationError` / Auto-converts "25" to int 25 / Nothing, runs fine

Q3: Which method converts a Pydantic model to a Python dict? → `model.to_dict()` / `model.dict()` / `model.serialize()` / `dict(model)`

Q4: `BaseSettings` in Pydantic is used for: → Database models / App configuration from env vars / API responses / Test mocking

Async Python
& Fast API
Fundamentals

Deloitte.

Async Python

& FastAPI Fundamentals

asyncio

FastAPI

async/await

Pydantic

GIL

Middleware

Multithreading

CORS

What We Will Cover

01

Threading, Multiprocessing & GIL

When to use what — concurrency vs parallelism

02

asyncio & async/await

Event loop, coroutines, tasks, gather — with real examples

03

FastAPI Introduction

What is it, why it's fast, project structure

04

Path, Query & Request Params

Routes, parameters, Pydantic request/response models

05

Dependency Injection

Reusable logic, DB sessions, auth

06

Middleware & CORS

Request/Response pipeline, cross-origin setup

SECTION 01

Threading, Multiprocessing & GIL

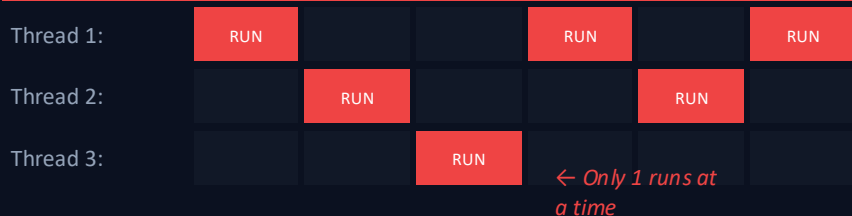
Concurrency vs Parallelism — choosing the right tool.

The GIL — Global Interpreter Lock



GIL = Global Interpreter Lock. A mutex that allows only ONE Python thread to execute bytecode at a time — even on multi-core CPUs. This means threads CAN'T run Python code in TRUE parallel.

WITH GIL — Threads take turns (not truly parallel)



BUT: GIL IS released during I/O operations (network, disk, sleep). So threading is still useful for I/O-bound tasks!



GIL NOT released for CPU-heavy work → use multiprocessing.



Which Tool for Which Job?

asyncio

BEST for FastAPI

Use for: I/O-bound + single thread

e.g. API calls, DB queries, web scraping

threading

GIL released for I/O

Use for: I/O-bound + multiple threads

e.g. Multiple network calls, file I/O

multiprocessing

Bypasses GIL

Use for: CPU-bound + true parallel

e.g. Image processing, ML, number crunching

Threading & Multiprocessing — Code Examples

Threading — I/O Bound Tasks

```
import threading
import time

def download(url):
    print(f"Downloading {url}")
    time.sleep(2)          # simulates network I/O
    print(f"Done: {url}")

# Run 3 downloads in parallel threads
threads = []
for url in ["site1.com", "site2.com", "site3.com"]:
    t = threading.Thread(target=download, args=(url,))
    threads.append(t)
    t.start()              # ← start each thread

for t in threads:
    t.join()               # ← wait for all to finish

# Total: ~2s (not 6s) because I/O releases GIL
```

Multiprocessing — CPU Bound Tasks

```
from multiprocessing import Pool

def heavy_compute(n):
    # Simulate CPU intensive work
    return sum(i*i for i in range(n))

# Create a pool of worker processes
with Pool(processes=4) as pool: # 4 CPU cores
    results = pool.map(
        heavy_compute,
        [1000000, 2000000, 3000000, 4000000]
    )
print(results) # Each ran on separate CPU core!

# True parallelism — GIL not an issue
# because each process has its OWN GIL
```



Quiz Time!

GIL, Threading & Multiprocessing

Q1

What does GIL stand for? → Global Instance Lock / Global Interpreter Lock / General Import Library / Global Index List

Q2

For downloading 100 files from the internet, which is BEST? → asyncio / threading / multiprocessing / All are same

Q3

GIL is released during: → CPU math operations / String parsing / I/O operations (network/disk) / All operations

Q4

Multiprocessing creates: → Threads / Coroutines / Separate processes with own GIL / A shared memory pool

SECTION 02

asyncio & async/await

Event loop, coroutines, tasks, gather — all demystified.

The Problem — Synchronous Code Waits & Wastes Time

 **Real-Life Analogy:** You go to a restaurant. Waiter takes your order and just STANDS there doing nothing until your food is ready (10 mins). Other customers wait. That is SYNCHRONOUS. A smart waiter takes your order and goes to serve others while food is prepared — that is ASYNC!

❌ Synchronous — Blocks & Waits

```
import time

def fetch_data(name):
    print(f"Fetching {name}...")
    time.sleep(3)      # ← BLOCKS for 3 sec
    print(f"{name} done!")

fetch_data("Users")    # waits 3s
fetch_data("Orders")  # then waits 3s
fetch_data("Products") # then waits 3s
# Total: 9 seconds ← wasteful!
```

✅ Asynchronous — Non-blocking

```
import asyncio

async def fetch_data(name):
    print(f"Fetching {name}...")
    await asyncio.sleep(3)  # ← yields control
    print(f"{name} done!")

async def main():
    await asyncio.gather(    # run ALL together!
        fetch_data("Users"),
        fetch_data("Orders"),
        fetch_data("Products"),
    )
    asyncio.run(main())

# Total: ~3 seconds only! 🚀
```

🕒 Timeline Comparison

Sync: Users 3s Orders 3s Products 3s = 9s

Async: All run concurrently → ~3s ✅

Asyncio Patterns – def() , await() , asyncio.sleep()

async def()

Used to define a coroutine function.
Example meaning:

Normal def → regular function
async def → async function that can
pause with await

Important:
Calling an async def function does not
run it fully by itself. It returns a
coroutine object that must be awaited or
scheduled.

💡 *Best for regular synchronous functions*

await

Used inside an async function.
Meaning:

"Pause here for now"
"Let the event loop do other work"
"Come back when this operation is
ready"

This is the key reason async is
efficient for waiting tasks like:

API calls
database calls
file/network I/O

💡 *Best when you want to do other work while
waiting*

asyncio.sleep(3)

Async version of sleeping.
Difference from time.sleep(3):

time.sleep(3) blocks the whole thread
asyncio.sleep(3) pauses only that
coroutine and lets others run

So await asyncio.sleep(3) is cooperative
waiting, not blocking.

💡 *Best for intentional delays — especially
asyncio.sleep() in async code*

Asyncn Patterns – gather(),create_task(),wait_for()

asyncio.gather()

Runs multiple coroutines concurrently and waits for all of them.

In the Code:

Users, Orders, and Products all start together
gather() waits until all 3 finish
returns their results in the same order
passed in

Use it when you want “start all, wait for all.”



Best when you need all results before continuing

asyncio.create_task()

```
# Schedule coroutine immediately (fire and go)
task = asyncio.create_task(fetch_data())

# Do other work while task runs...
process_something_else()

result = await task # collect later

gather() = run many and wait for all together
create_task() = start something now, manage it separately
```



Best when you want to do other work while waiting

asyncio.wait_for() — Timeout

```
# Cancel if takes too long
try:
    result = await asyncio.wait_for(
        slow_api_call(),
        timeout=5.0 # cancel after 5 sec
    )
except asyncio.TimeoutError:
    print("API too slow!")
```



Best for I/O-bound work like API and database calls

Key Asyncio Terms and Comparison

<code>async def</code>	Declares a coroutine function (returns a coroutine object)
<code>await</code>	Pauses coroutine, yields control to event loop until result ready
<code>asyncio.run()</code>	Entry point — creates event loop and runs the coroutine
<code>asyncio.gather()</code>	Run multiple coroutines concurrently, wait for all
<code>asyncio.create_task()</code>	Schedule coroutine to run soon (non-blocking)
<code>asyncio.sleep(n)</code>	Async wait — yields control (unlike <code>time.sleep()</code>)

Aspect	asyncio	threading	multiprocessing
Concurrency type	Single-threaded	Multi-threaded	Multi-process
I/O bound	★ ★ ★ Best	★ ★ Good	★ OK
CPU bound	✗ Bad	✗ Bad (GIL)	★ ★ ★ Best
Memory	Shared	Shared	Separate
Complexity	Medium	Medium	High
GIL affected	No (single thread)	Yes (CPU tasks)	No (own process)



Quiz Time!

Async Python — async/await & asyncio

Q1 | What keyword pauses a coroutine and yields control to the event loop? → return / yield / await / suspend

Q2 | asyncio.gather() is used to: → Run one coroutine / Run multiple coroutines concurrently / Wait for one task / Create a new thread

Q3 | Which of these BLOCKS the event loop? → await asyncio.sleep(1) / asyncio.gather() / time.sleep(1) / asyncio.create_task()

Q4 | async def makes a function return: → A normal value / A coroutine object / A thread / A generator

SECTION 03

FastAPI Introduction

What is it? Why is it fast? How to structure a project?

What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on **Starlette** (web framework) + **Pydantic** (data validation) + supports **async/await** natively. It is used to build APIs quickly and cleanly.



Very Fast

Fast API is fast because it is built on modern async technology. It handles many requests efficiently



Type Hints

We can write clear input and output types. This helps the editor, reduces mistakes, and makes code easier to understand



Auto Docs

Fast API automatically creates API documentation for us. So we do not need to manually write every endpoint description



Async Native

Fast API works very well with async and await. So it is great for APIs that call databases, other APIs, or external services



Auto Validation

It checks incoming data automatically. If a user sends wrong data, Fast API can catch it and return a clear error



Standards Based

It follows common web API standards like Open API and JSON Schema, so it works well with other tools

```
# Install FastAPI
pip install fastapi uvicorn[standard] pydantic
```

```
# uvicorn = ASGI server to run FastAPI
```

FastAPI Project Structure — Best Practice

```

my_fastapi_app/
├── main.py                ← app entry point, create
FastAPI(), include routers
├── requirements.txt       ← dependencies
├── app/
│   ├── __init__.py
│   ├── models/           ← Pydantic models (schemas (data
base tables)
│   │   ├── user.py       ← UserCreate, UserResponse, etc.
│   │   └── product.py
│   ├── routers/          ← Define the Endpoints..
│   │   ├── users.py      ← GET /users, POST /users, etc.
│   │   └── products.py
│   ├── services/         ← Business logic (keep routers
thin)
│   │   └── user_service.py
│   ├── dependencies.py    ← Shared dependencies (DB session,
auth)
│   ├── middleware.py      ← Custom middleware
│   └── config.py          ← Settings (from env vars via
Pydantic)
└── tests/
    └── test_users.py

```

main.py — entry point

```
from fastapi import FastAPI
from app.routers import users, products
from app.middleware import add_middleware

app = FastAPI(
    title="My API",
    description="FastAPI example",
    version="1.0.0",
)

# include routers
app.include_router(users.router)
app.include_router(products.router)

# add middleware
add_middleware(app)
```

How to run

```
# Development (auto-reload on save)
uvicorn main:app --reload

# Production
uvicorn main:app --host 0.0.0.0 --port 8000

# Then open browser:
# http://localhost:8000/docs      ← Swagger UI
# http://localhost:8000/redoc    ← ReDoc
```

SECTION 04

Request/Response

Path params, query params, and Pydantic models for data.

Path Params & Query Params — Receiving Data from URL

URL Anatomy:

`https://api.com/users/42?page=2&limit=10`

↑ path param (user_id=42)

↑ query params

```
from fastapi import FastAPI, Query, Path
app = FastAPI()

# — PATH PARAMETER — part of the URL path —————
@app.get("/users/{user_id}")
async def get_user(
    user_id: int          # ← FastAPI auto-validates: must be int!
):
    return {"user_id": user_id}
# URL: GET /users/42          → {"user_id": 42}
# URL: GET /users/abc        → 422 Unprocessable Entity (not int!)

# — QUERY PARAMETERS — after the ? —————
@app.get("/users")
async def list_users(
    page: int = 1,          # ← optional, default=1
    limit: int = 10,        # ← optional, default=10
    search: str | None = None # ← optional, can be None
):
    return {"page": page, "limit": limit, "search": search}
# URL: GET /users          → {"page":1,"limit":10,"search":null}
# URL: GET /users?page=2&search=alice → {"page":2,"limit":10,"search":"alice"}
```

Path param = what resource

Query param = how to view/filter that resource

Use path params when the value is required to locate the item. Use query params when the value is optional or changes the response behavior.

Advanced Validation

```
@app.get("/items/{item_id}")
async def get_item(
    # Path() adds validation to path param
    item_id: int = Path(
        ...,          # required (no default)
        ge=1,          # >= 1
        le=1000,       # <= 1000
        title="Item ID",
    ),
    # Query() adds validation to query param
    q: str = Query(
        default=None,
        min_length=3,
        max_length=50,
    )
):
    return {"item_id": item_id, "q": q}
```

Auto Type Coercion

`?page="2"` → FastAPI converts string "2" to int 2 automatically!

Pydantic Models — Request Body & Response Models

 **Request body** = data the client sends (POST/PUT/PATCH).  **Response model** = what FastAPI returns & validates. Both use Pydantic BaseModel.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field, EmailStr
from typing import Optional
from datetime import datetime

app = FastAPI()

# — MODELS (schemas/app/models/user.py) —————
class UserCreate(BaseModel):    # what client sends
    name: str = Field(..., min_length=2, max_length=50)
    email: EmailStr             # validates email format!
    age: int = Field(..., ge=18) # must be 18+
    city: Optional[str] = None  # optional

class UserResponse(BaseModel):  # what we return
    id: int
    name: str
    email: str
    created_at: datetime

    class Config:
        from_attributes = True    # allows ORM model → Pydantic

# — ROUTE —————
@app.post(
    "/users",
    response_model=UserResponse,    # ← FastAPI filters & validates response!
    status_code=201,               # ← HTTP 201 Created
)

async def create_user(user: UserCreate):    # ← body auto-parsed & validated
    # user.name, user.email, user.age etc are validated already!
    new_user = save_to_db(user)            # returns DB record
    return new_user                        # auto-serialized using UserResponse
```

Why Separate Models?

UserCreate

What client sends — no id, no timestamps. Has password.

UserResponse

What we return — has id, created_at. Never password!

UserUpdate

PATCH — all fields optional (for partial updates)

UserInDB

Internal model — has hashed_password field

Quick HTTP Codes: 200 OK 201 Created 204 No Content 400 Bad Request 401 Unauth 404 Not Found 422 Validation 500 Server Error



Quiz Time!

FastAPI Routes & Pydantic Models

Q1 In FastAPI, `/users/{user_id}` — `user_id` is a: → Query parameter / Path parameter / Request body / Header

Q2 What happens if you send `?page=abc` when `page: int` is declared? → Returns `page=abc` / Converts to 0 / Returns 422 Validation Error / Crashes server

Q3 `response_model` in FastAPI route is used to: → Validate input / Filter & validate what is returned to client / Define database schema / Add authentication

Q4 To make a query parameter optional in FastAPI, you: → Use `Optional[str] = None` / Use `query=optional` / Wrap in `try/except` / Do nothing — all are optional

SECTION 05

Dependency Injection

Reusable logic wired in cleanly — DB, auth, pagination.

Dependency Injection — Reusable Logic Made Easy

💡 **DI** = Instead of writing the same logic (auth check, DB connection, pagination) in every route, you define it ONCE as a dependency. FastAPI injects it automatically using **Depends()**. Dependency Injection solves this problem by injecting the dependencies into the class from an external source, rather than having the class create them.

```
from fastapi import FastAPI, Depends, HTTPException, Header
from typing import Annotated

app = FastAPI()

# — DEPENDENCY 1: Pagination (reused in many routes) —————
def get_pagination(skip: int = 0,):
    return {"skip": skip, "limit": limit} limit: int = 10

# — DEPENDENCY 2: Fake Auth (check token in header) —————
def verify_token(x_token: str = Header(...)):
    if x_token != "secret-token":
        raise HTTPException(status_code=401, detail="Invalid token")
    return x_token

# — DEPENDENCY 3: DB Session (yield pattern — ensures cleanup) —
def get_db():
    db = DatabaseSession() # open connection
    try:
        yield db # ← 'yield' makes it a context manager!
    finally:
        db.close() # ALWAYS closes, even on error

# — USE DEPENDENCIES IN ROUTES —————
@app.get("/users")
async def list_users(
    pagination: dict = Depends(get_pagination), # ← inject!
    token: str = Depends(verify_token), # ← inject!
    db = Depends(get_db), # ← inject!
):
    return db.get_users(**pagination)
```

Benefits of DI



Reusable

Write once, use in 100 routes



Clean routes

Routes stay thin & readable



Testable

Easy to mock dependencies in tests



Auto-injected

FastAPI handles the calling order



Auth patterns

Perfect for token/session checks



DB sessions

yield pattern ensures conn cleanup



Quiz Time!

Dependency Injection

Q1

In FastAPI, Depends() is used to: → Create database tables / Inject reusable logic into routes / Define request body / Add middleware

Q2

The yield pattern in a dependency is used for: → Returning multiple values / Ensuring cleanup (e.g., DB connection close) / Making it async / Caching results

Q3

If verify_token raises HTTPException(401), what happens? → Route runs anyway / FastAPI returns 401 to client immediately / Server crashes / Token is ignored

Q4

Which is a valid use case for Dependency Injection? → Defining Pydantic models / Reusable pagination (skip, limit) / Setting HTTP status code / Defining URL paths

SECTION 06

Middleware & CORS

Intercepting requests, adding headers, and cross-origin setup.

Middleware — Intercept Every Request & Response

Middleware runs on EVERY request & response — like a pipeline layer between client and your routes.



Request →

```
from fastapi import FastAPI, Request
import time

app = FastAPI()

# — CUSTOM MIDDLEWARE using @app.middleware —————
@app.middleware("http")
async def add_timing_header(request: Request, call_next):
    start = time.time()

    # ← before route runs (intercept request)
    print(f"Incoming: {request.method} {request.url}")

    response = await call_next(request)  # ← call the actual route

    # → after route runs (intercept response)
    duration = time.time() - start
    response.headers["X-Process-Time"] = str(duration)
    print(f"Completed in {duration:.3f}s")

    return response  # ← send back to client
```

← Response

Middleware Use Cases



Logging

Log every request URL, method, status, duration



Timing

Measure how long each route takes



Security Headers

Add X-Frame-Options, HSTS to all responses



Rate Limiting

Block IPs making too many requests



Request ID

Track requests with unique ID



Compression

GZipMiddleware for smaller responses

CORS — Cross-Origin Resource Sharing



What is CORS?

Browser security blocks requests from one origin to another by default. Example: your React app on `localhost:3000` tries to call FastAPI on `localhost:8000` — browser BLOCKS it! CORS tells the browser which origins are allowed.

React App
`localhost:3000`



✗ Browser
Blocks!



FastAPI
`localhost:8000`

With CORS middleware configured → Allowed!

CORS — Cross-Origin Resource Sharing

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# — ADD CORS MIDDLEWARE —————
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",      # React dev
        "https://myapp.com",          # Production frontend
        # Use ["*"] to allow ALL origins (dev only – not safe in prod!)
    ],
    allow_credentials=True,           # allow cookies, auth headers
    allow_methods=["GET", "POST", "PUT", "DELETE", "PATCH"], # or ["*"]
    allow_headers=["*"],              # which headers are allowed
)

@app.get("/users")
async def get_users():
    return [{"id": 1, "name": "Alice"}]

# Now React on localhost:3000 CAN call this API!
# FastAPI adds "Access-Control-Allow-Origin" header to responses
```

CORS Settings

allow_origins

Which URLs can call your API

allow_methods

Which HTTP methods (GET, POST...)

allow_headers

Which request headers allowed

allow_credentials

Allow cookies & auth headers

"*" wildcard

Allow everything (dev only!)

⚠️ Never use ["*"] for allow_origins in production! Always list specific domains.



Quiz Time!

Middleware & CORS

Q1

Middleware runs: → Only on error / Only for POST requests / On every request and response / Only once at startup

Q2

CORS is needed when: → Frontend and backend are on the same port / Frontend and backend are on different origins (domain/port) / Using HTTPS / Connecting to a database

Q3

In CORSMiddleware, `allow_origins=["*"]` means: → Block all origins / Allow only localhost / Allow all origins (unsafe for production!) / Require credentials

Q4

The `call_next(request)` in middleware: → Creates a new request / Calls the actual route handler and gets the response / Cancels the request / Logs the request

Thank You